

VLA-Fabric: Communication-Efficient Coordination for Scalable Networks of VLA Agents

Anonymous Authors

Abstract—Vision-language-action (VLA) models connect visual and language understanding with executable robot actions, but current systems scale poorly across multiple arms. Single- and multi-arm policies are typically built for a fixed embodiment, so changing the team size often requires constructing and training another centralized policy. Meanwhile, separately deployed VLA-controlled arms and robots usually operate in isolation, without communicating to coordinate physical actions. Enabling communication among such policies offers a path toward more scalable and interactive multi-robot organization.

In this paper, we present VLA-Fabric, a distributed organization of cooperating VLA agents. Each agent controls one arm from local observations and outputs local actions; teams of different sizes are formed by adding communicating agents rather than enlarging a fixed joint-action model. The agents coordinate through three intermediate-representation paths. To reduce communication overhead and support time-sensitive control loops, we introduce the *next-synchronization post-residual* (NSPR) metric and an NSPR-guided communication-layer screening algorithm. Across paired evaluations on two- to four-agent tasks, NSPR-8 reduces bidirectional communication by 88.8–89.0% relative to Raw projected communication while keeping task success within 3.5 percentage points of the reference. Beyond these efficiency gains, VLA-Fabric establishes explicit inter-policy communication as a practical systems interface for networked VLA agents.

I. INTRODUCTION

Artificial intelligence is increasingly moving beyond digital content toward direct interaction with the physical world. Robotic foundation models seek to turn large-scale learned representations into perception, reasoning, and control capabilities that operate in unstructured environments. Vision-language-action (VLA) policies and related embodied models are central to this transition because they connect semantic understanding to closed-loop physical behavior.

Among these models, a VLA policy maps visual observations, language instructions, and robot state to executable actions. A typical system combines a visual encoder, a vision-language or policy backbone, and an action decoder that predicts individual controls or an action chunk. RT-2 [1] casts robot actions as tokens within a vision-language model, while OpenVLA [2] provides an open pretrained VLA for downstream robot adaptation. Octo [3] learns a generalist transformer policy across heterogeneous robot data, and π_0 [4] uses flow matching to generate continuous action chunks. Together, these systems illustrate the rapid expansion of VLA architectures, training objectives, and manipulation capabilities.

Despite this diversity, most work follows the same organizational pattern: it strengthens one centralized policy from differ-

ent aspects, such as scaling pretraining, enriching spatial representations, and broadening open-world generalization. For example, Xiaomi-Robotics-1 pretrains on more than 100,000 hours of real-world trajectories [5]; SpatialVLA injects 3D information and spatially adaptive action representations [6]; and $\pi_{0.5}$ combines heterogeneous robot data with high-level semantic prediction for open-world generalization [7]. Multi-arm methods extend the same centralized pattern. ET-VLA [8] adapts an autoregressive VLA to bimanual embodiments, while TwinVLA [9] couples two arm-specific VLA streams through joint attention. Both improve multi-arm control, but the observations, intermediate interactions, and joint action are still resolved inside one computation graph configured for a particular embodiment and arm count.

Communication is already a first-class design variable in other multi-agent learning systems. CommNet [10] learns continuous communication jointly with control, DIAL [11] trains differentiable inter-agent messages, and TarMAC [12] uses attention to target information to relevant peers. Communication also organizes language-model agents: CAMEL [13] studies cooperation through role-playing conversations, and AutoGen [14] provides a programmable multi-agent conversation framework. Embodied multi-agent systems have also coordinated task allocation, motion planning, and compositional constraints [15], [16]. These methods organize tasks or execution, but they do not establish learned communication among independently executable, action-producing VLA policies. Such communication, particularly through VLA intermediate representations, remains largely unexplored.

This centralized organization creates two system-level limitations that cannot be addressed simply by strengthening another isolated policy. First, it scales poorly: the number and ordering of observations, robot states, and action dimensions are fixed when the policy is constructed. Moving from one or two arms to a larger team changes the model interface and commonly creates a new multi-agent learning problem; fixed bimanual benchmarks such as RoboTwin 2.0 illustrate this coupling between policy structure and arm count [17]. Second, a standalone VLA has no learned protocol for telling another VLA what it perceives, what context it needs, or what action it intends to execute. Independently deployed policies therefore remain isolated, while a centralized policy offers no natural computation or communication boundary between arm-side devices.

We target this organizational bottleneck by moving multi-arm VLA control from a fixed centralized graph to a distributed *one-arm-one-agent* abstraction. As illustrated in

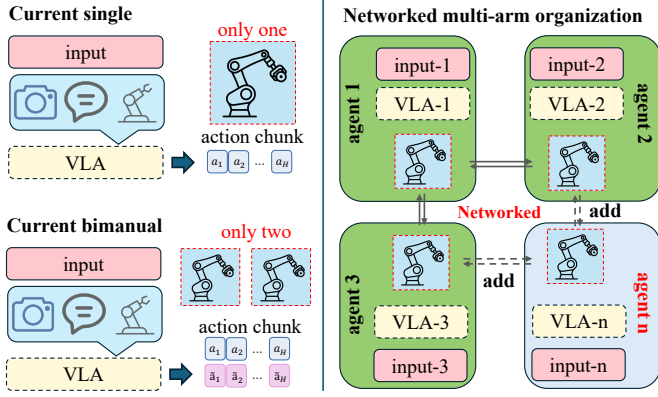


Fig. 1. From fixed-arm centralized policies to VLA-Fabric. Existing VLAs bind a predetermined set of observation and action streams inside one model. VLA-Fabric assigns one complete VLA agent to each arm and exposes their coordination as an explicit communication interface, so the organization grows by adding agents rather than enlarging a joint-action model.

Fig. 1, each arm retains a complete VLA with its own observations, policy computation, and local action output. Agents coordinate by exchanging explicit intermediate-representation messages rather than by widening a shared joint-action head. Adding an arm consequently adds an agent and its communication edges, while removing an arm does not redefine the action space of the remaining agents.

The broader significance of VLA-Fabric is that cross-policy communication becomes a first-class systems interface rather than an internal operation hidden inside a centralized model. We call the resulting independently executable policies *networked VLA agents*. This interface makes what to communicate, when to communicate, and with whom to communicate explicit and optimizable, while allowing the organization to reduce to one agent or compose larger teams.

Making coordination explicit also exposes a systems bottleneck. Intermediate representations that were read locally inside a centralized graph must now be synchronized across processes and devices, while closed-loop manipulation imposes strict communication and planning deadlines. Similar bandwidth bottlenecks arise in multi-device transformer inference [18], but robotic control additionally couples communication quality to action stability. We therefore compress common synchronization into a hidden workspace and define the *next-synchronization post-residual* (NSPR) metric. Our communication-layer screening algorithm uses NSPR to identify low-utility synchronization layers while retaining closed-loop safeguards.

Across three bimanual tasks and three-agent Stack Cube, removing all communication leaves only 1.5–20.0% success, confirming that the exchanged representations carry task-critical coordination. On handover box, VLA-Fabric reaches 75.0% success versus 76.0% for its centralized TwinVLA reference [9]; on four-agent Take Photo, it reaches 96.5% versus 20.0% for the published RoboFactory DP baseline [16]. Across two- to four-agent tasks, one fixed eight-layer profile reduces communication by 88.8–89.0% relative to Raw with-

out reducing success by more than 3.5 percentage points.

This paper makes three contributions:

- We formulate communication among complete action-producing VLA policies as a systems problem and instantiate it with VLA-Fabric, a one-arm-one-agent organization that preserves local observation and action ownership while composing larger teams through explicit communication.
- We design a three-path intermediate-representation interface for communication among independently executable agents and implement both centralized training and process- and device-isolated inference.
- We develop hidden-workspace preprocessing and an NSPR-guided layer-screening algorithm that jointly reduce communication while protecting closed-loop behavior.

II. PRELIMINARIES AND RELATED WORK

We first define the execution units used in VLA-Fabric and then distinguish its communication interface from centralized multi-arm policies, learned multi-agent communication, and partitioned DNN inference. The comparison focuses on where the control boundary lies, which component owns each local action, and what information crosses the communication interface.

VLA Execution Model and Terminology. At control time t , a VLA maps visual observations o_t^v , proprioceptive state q_t^p , and task instruction p to an H -step action chunk $a_i^{t:t+H-1}$ through a visual encoder, a policy backbone, and an action decoder [1]–[4]. Action chunking amortizes policy inference over multiple environment steps. After each chunk is executed, the next planning round conditions on the resulting visual observations and proprioceptive state. We call one cycle of local policy computation, inter-agent representation exchange, and action decoding a *planning round*.

A *complete VLA agent* owns its local observations and policy state and produces one arm’s action. A *model partition*, by contrast, implements only part of one computation graph and owns neither an independent control objective nor a local action. Local observations remain private, whereas shared task context includes information such as the instruction or a global view. A synchronization layer is a transformer boundary at which agents refresh an exchanged representation, and the active boundaries form a synchronization profile. These definitions distinguish communication among action-producing agents from ordinary model parallelism.

Centralized Multi-Arm Policies. Multi-arm VLAs generally retain an embodiment-specific centralized interface. ET-VLA adapts an autoregressive single-arm policy to emit the longer action-token sequence required by two robots [8]. Twin-VLA instead composes two pretrained single-arm branches through joint attention and shared representation processing [9]. Although their action-generation mechanisms differ, both resolve cross-arm interaction and action composition inside one policy configured for a fixed bimanual embodiment.

TABLE I
SYSTEM-LEVEL POSITIONING OF VLA-FABRIC AND NEIGHBORING RESEARCH DIRECTIONS.

Method family	Execution unit	Exchanged content	Local action ownership	Team organization	Communication objective
Centralized multi-arm VLA [8], [9], [19]	Embodiment-specific joint policy	No exposed inter-policy message	Joint/centrally decoded	Fixed at construction	Not explicit
Multi-robot planning [15], [16]	Robot/controller	Plan, state, task, or trajectory	Local	Configurable	Feasibility and coordination
MARL communication [10]–[12]	Task-specific learned agents	Learned messages	Local	Task-dependent	Expected task reward
Partitioned DNN inference [18]	Model partitions	Activations or K/V	None	Fixed model graph	Latency and bandwidth
VLA-Fabric	Complete VLA agents	Selected intermediate representations	Local	Agent-wise composition	Closed-loop utility and network cost

This organization is effective when all observations, computation, and actions reside on one platform. The limitation relevant here is organizational: arm count and role ordering remain part of the policy interface, adding another arm changes the observation and action structure, and no transport boundary is exposed between complete arm-local policies. VLA-Fabric preserves local action ownership and makes cross-policy interaction a separately measurable communication interface.

Communication in Multi-Agent Systems. Multi-agent systems exchange task assignments, trajectories, states, natural language, or learned latent messages. Multi-robot planners and language-model agents communicate above the action policy to coordinate tasks, plans, and constraints [13]–[15]. Their messages primarily determine what agents should do rather than expose how a pretrained policy generates its next action.

MARL methods such as CommNet, DIAL, and TarMAC instead optimize differentiable messages jointly with task-specific controllers [10]–[12]. They establish that task reward can shape compact inter-agent communication, but the controller and message semantics are normally learned together for a particular multi-agent problem. VLA-Fabric starts from complete action-producing VLA policies and exposes selected intermediate representations during closed-loop action generation.

Communication-Efficient Distributed Inference. Distributed model execution partitions one computation graph across devices and exchanges activations or K/V states. GPipe and Megatron-LM establish pipeline- and tensor-parallel execution, while ASTRA directly targets communication-efficient transformer inference [18], [20], [21]. Their optimization commonly targets throughput, latency, or output fidelity under a shared model input. VLA-Fabric instead connects complete policies that receive different private observations and own separate local actions. Representation error also enters a physical feedback loop, where one action deviation changes future observations and can accumulate into grasp, transfer, or release failure. Tensor distortion and throughput are therefore insufficient alone; communication must also be evaluated against action divergence and paired closed-loop success.

Positioning of VLA-Fabric. VLA-Fabric occupies the systems layer between high-level multi-robot coordination

and low-level partitioned inference. It neither allocates tasks nor splits one monolithic model. Instead, it defines an explicit transport contract among complete action-producing VLA policies, making message type, topology, representation, codec, refresh interval, and synchronization layer independently measurable and optimizable. The resulting objective is joint: network cost must be reduced without disrupting closed-loop task behavior. Here, agent-wise composition means forming a team by adding complete VLA agents. Training is team-size specific, but a fixed-size team can learn multiple tasks jointly. Table I summarizes this positioning.

III. VLA-FABRIC ARCHITECTURE

VLA-Fabric assigns action generation to arm-local policies and exposes cross-arm coordination through an explicit communication interface. Each arm runs a complete VLA that owns its local observation and action, while collaboration occurs through selected representations exchanged over a communication graph. This section first defines the agent organization and communication topology, then details the three-message interface and its execution within one planning round, and finally describes the centralized-training and distributed-execution boundary together with the communication cost model.

A. Agent Organization and Communication Graph

VLA-Fabric organizes N networked VLA agents $\mathcal{A} = \{A_1, \dots, A_N\}$ through a directed communication graph $G = (\mathcal{A}, \mathcal{E})$. An edge $(j, i) \in \mathcal{E}$ permits agent j to send coordination messages to agent i , whose incoming neighbor set is

$$\mathcal{N}(i) = \{j \mid (j, i) \in \mathcal{E}\}. \quad (1)$$

At planning round t , agent i receives local visual observation o_i^t , local proprioception q_i^t , task instruction p , and messages $\mathcal{R}_i^t$ from $\mathcal{N}(i)$. It emits only its local action chunk:

$$a_i^{t:t+H-1} = \pi_i(o_i^t, q_i^t, p; \mathcal{R}_i^t). \quad (2)$$

At execution, the arm-specific outputs are dispatched together to their corresponding controllers; no learned module recombines them into a centralized joint action. Consequently, each agent’s action space remains independent of team size, and only explicitly defined coordination messages cross agent

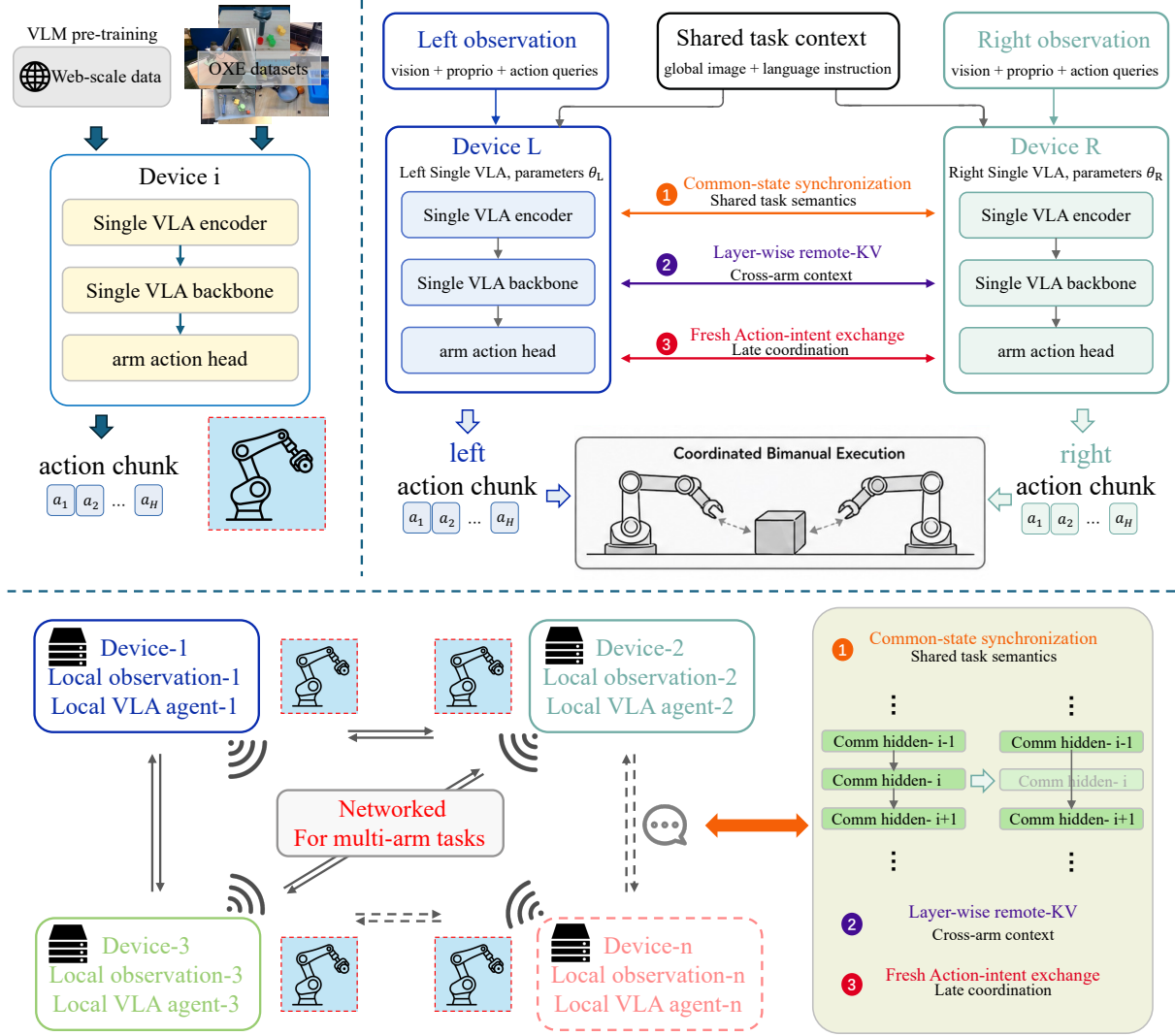


Fig. 2. VLA-Fabric architecture. Top: pretrained single-arm VLAs are initialized as independently executable arm-local agents that coordinate through common-state synchronization, layer-wise remote K/V, and action-intent exchange. Bottom: the same three-message interface connects an extensible set of device-resident agents while each agent retains its local observation and action policy.

boundaries. For $N = 1$, $\mathcal{E}$ is empty and the organization reduces to the underlying single-arm policy.

Figure 2 illustrates the two-agent construction. Its links are symmetric, while our three- and four-agent experiments use a role-ordered all-to-all logical graph. Under unicast transport, this graph contains $N(N-1)$ directed peer relations and gives each agent $O(N)$ incoming peer context. Other communication graphs can be used without changing the arm-local policy interface.

B. Three-Message Coordination Interface and Aggregation

The boundary in Fig. 2 exposes three representations at different policy locations. The *common semantic message* $\mathcal{M}^{\text{com}}$ synchronizes task language and shared scene context. The *private context message* $\mathcal{M}^{\text{kv}}$ carries peer-specific keys and values while keeping local queries and raw observations private. The *action-intent message* $\mathcal{M}^{\text{act}}$ communicates compact readout tokens immediately before local action decoding.

Common semantics. At a selected transformer stage ℓ , agent i exposes a common-path representation $M_{\text{com},i}^\ell$ containing shared task and scene information required by subsequent computation. Participating agents aggregate these representations according to the communication graph:

$$\bar{M}_{\text{com},i}^\ell = \text{Agg}_{\text{com}}(\{M_{\text{com},j}^\ell : j \in \mathcal{N}(i) \cup \{i\}\}). \quad (3)$$

For the all-to-all topology evaluated here, Agg_{com} is symmetric averaging across participating agents.

Private K/V. At layer ℓ , each agent’s private hidden state produces $(Q_i^\ell, K_i^\ell, V_i^\ell)$. Received peer keys and values are concatenated in a fixed role order,

$$\hat{K}_{\mathcal{N}(i)}^\ell = \text{Concat}_{j \in \mathcal{N}(i)}^{\text{role}} K_j^\ell, \quad \hat{V}_{\mathcal{N}(i)}^\ell = \text{Concat}_{j \in \mathcal{N}(i)}^{\text{role}} V_j^\ell. \quad (4)$$

The local query attends to common, local-private, and remote-private context:

$$\tilde{H}_i^{\ell+1} = \text{Attn}(Q_i^\ell, [K_{c,i}^\ell, K_i^\ell, \hat{K}_{\mathcal{N}(i)}^\ell], [V_{c,i}^\ell, V_i^\ell, \hat{V}_{\mathcal{N}(i)}^\ell]). \quad (5)$$

Queries never cross the private channel, and sender ordering preserves peer identity when several agents contribute context.

Action intent. Immediately before decoding, agent i extracts a compact intent token z_i from its late private policy state. The current implementation concatenates all available tokens in global sender order and applies a receiver-specific learned linear projection:

$$\tilde{z}_i = \phi_i^{\text{act}} \left(\text{Concat}_{j \in \mathcal{N}(i) \cup \{i\}}^{\text{role}} z_j \right). \quad (6)$$

The local decoder conditions its action chunk on $\tilde{z}_i$. Unlike common synchronization and private K/V, which act inside the transformer stack, action intent is exchanged once near the decoder boundary and conveys late-stage information about peers' imminent control decisions. Because the three channels enter the policy at different stages, they can use channel-specific representations, coding schemes, and refresh policies. In this work, communication-layer screening is applied to the dominant common path.

C. One Planning Round

A planning round executes the three paths in policy order. Each agent first encodes its local visual observation, proprioception, and shared instruction. At selected transformer boundaries, agents refresh the common workspace; local queries then attend to role-ordered private K/V received from their neighbors. Immediately before action decoding, agents exchange and fuse compact action-intent tokens. After local decoding, the arm-specific commands are executed synchronously, and the next planning round starts from the resulting observations. Communication errors can therefore affect both the current action and the future state distribution.

During a handover, common synchronization aligns task and scene semantics, private K/V supplies peer-specific visual and state context, and action intent provides late-stage information associated with holding, transferring, or releasing. The three paths intervene at different stages of action generation rather than transmitting redundant copies of the same information.

D. Centralized Training, Distributed Execution, and Cost Model

During training, the communication operators are evaluated within a unified computation graph, allowing gradients to propagate across agents without emulating network transport. At deployment, each agent maps directly to a separate process and device, performs perception and action decoding locally, and exchanges only the three named representations through an explicit communication backend. This design simplifies centralized training and provides a model-native partition boundary for distributed inference: local observations and control remain on-device, while cross-agent coordination is isolated into measurable communication messages.

For message group $g \in \{\text{com}, \text{kv}, \text{act}\}$, let c_{ji}^t denote transmitted bytes on edge (j, i) and $x_{ji}^t \in \{0, 1\}$ indicate

whether the group is active. Total traffic in planning round t is

$$C_t = \sum_{(j,i) \in \mathcal{E}} \sum_g x_{ji}^t c_{ji}^t, \quad (7)$$

and episode traffic is $\sum_t C_t$. This formulation separates three sources of communication cost: topology $\mathcal{E}$, channel activation x_{ji}^t , and per-message payload c_{ji}^t . These terms expose independent system controls for graph scheduling, message selection, representation compression, and synchronization-layer screening.

IV. COMMUNICATION-EFFICIENT LAYER SCREENING

Making inter-agent coordination explicit also makes its communication cost directly observable. We first replace the redundant operator-level common path with a compact hidden workspace, and then formulate communication-layer selection as a constrained cost-reduction problem. Based on this formulation, we introduce NSPR-guided screening to remove low-utility synchronization layers while retaining closed-loop behavioral safeguards.

A. Common-Path Communication Preprocessing

The three coordination paths have highly asymmetric communication costs. Under Raw, one planning round transfers 148.877 MiB bidirectionally: common synchronization contributes 142.471 MiB (95.7%), private K/V contributes 6.403 MiB, and action intent contributes approximately 3.5 KiB. We therefore focus communication preprocessing and layer screening on the dominant common path while keeping the other two paths enabled.

Raw directly externalizes the operator-level common-path dependencies of the original cross-agent computation and serves as the uncompressed communication reference. We evaluated compact alternatives for this path and found that exchanging the layer-boundary common hidden state preserves closed-loop performance while sharply reducing both payload and synchronization operations. We use this hidden workspace as the initial representation for the layer-screening procedure described next.

B. Layer-Screening Objective

After hidden-workspace preprocessing, let $\mathcal{L} = \{0, \dots, L-1\}$ be the candidate transformer layers, $\mathcal{S}_0 \subseteq \mathcal{L}$ the profile from which screening starts, and $\mathcal{S}_{\text{ref}} \subseteq \mathcal{L}$ the fixed profile used for behavioral comparison. Let $\mathcal{D}$ denote the paired development conditions and $P_{\mathcal{D}}(\mathcal{S})$ the empirical success rate of profile $\mathcal{S}$ on them. Keeping the reference fixed prevents accepted errors from becoming the baseline for later removals. In the reported experiments, both profiles are initialized to the 12-layer even profile $\{0, 2, \dots, 22\}$. We seek a low-cost profile whose closed-loop performance remains within tolerance ϵ of the fixed reference:

$$\min_{\mathcal{S} \subseteq \mathcal{S}_0} C(\mathcal{S}) \quad (8)$$

$$\text{s.t. } P_{\mathcal{D}}(\mathcal{S}) \geq P_{\mathcal{D}}(\mathcal{S}_{\text{ref}}) - \epsilon. \quad (9)$$

Exhaustive subset evaluation is impractical because every candidate requires closed-loop rollouts. Our screening algorithm therefore uses fixed-state replay metrics, including NSPR, to rank drop-one candidates and reserves paired rollouts for a small safety-gated set.

An immediate hidden-state difference is not a reliable measure of layer importance. Removing one synchronization can create a large local perturbation that the next active synchronization almost completely repairs. Conversely, a small local deviation can persist through later layers and reach the action decoder. Ranking layers only at the removal point would therefore overvalue temporary discrepancies and miss low-amplitude but behaviorally persistent errors.

For parent profile $\mathcal{S}$, candidate $\mathcal{S}^{-\ell} = \mathcal{S} \setminus \{\ell\}$ removes synchronization at layer ℓ . On fixed state x , its residual perturbation at downstream layer k is

$$E_{\ell \rightarrow k} = \frac{\|H_k(\mathcal{S}^{-\ell}) - H_k(\mathcal{S})\|_F}{\|H_k(\mathcal{S})\|_F + \varepsilon}. \quad (10)$$

The behavioral utility combines action-chunk and action-token divergence relative to the fixed reference. For replay state x and drop-one candidate $\mathcal{S}^{-\ell}$, the implementation uses

$$D_{\text{act}}^{\ell}(x) = \frac{\|a(x; \mathcal{S}^{-\ell}) - a(x; \mathcal{S}_{\text{ref}})\|_2}{\|a(x; \mathcal{S}_{\text{ref}})\|_2 + \varepsilon}, \quad (11)$$

$$D_{\text{tok}}^{\ell}(x) = \frac{1}{2} \frac{\|z(x; \mathcal{S}^{-\ell}) - z(x; \mathcal{S}_{\text{ref}})\|_2}{\|z(x; \mathcal{S}_{\text{ref}})\|_2 + \varepsilon} + \frac{1}{2} \left(1 - \frac{\langle z(x; \mathcal{S}^{-\ell}), z(x; \mathcal{S}_{\text{ref}}) \rangle}{\|z(x; \mathcal{S}^{-\ell})\|_2 \|z(x; \mathcal{S}_{\text{ref}})\|_2 + \varepsilon} \right), \quad (12)$$

$$U_{\ell}(x) = D_{\text{act}}^{\ell}(x) + \frac{1}{2} D_{\text{tok}}^{\ell}(x), \quad (13)$$

where z denotes the paired action-intent tokens before local decoding. One statistic is insufficient because a layer can appear unimportant on average yet cause a tail failure in one task phase. We therefore partition replay states into early, middle, and late phases. Let $Q_{0.95}(Y \mid p)$ denote the empirical 95th percentile of quantity Y over replay states in phase p . This upper-tail statistic captures consistently large perturbations without allowing one extreme sample to dominate the ranking. We assign each candidate four metrics:

$$r_{\ell} = \max_p Q_{0.95}(E_{\ell \rightarrow k^+} \mid p), \quad u_{\ell} = \max_p Q_{0.95}(U_{\ell} \mid p), \quad (14)$$

$$o_{\ell} = \max_x U_{\ell}(x), \quad v_{\ell} = \max_p Q_{0.95}(D_{\text{act}}^{\ell} \mid p). \quad (15)$$

Here k^+ is the next active synchronization layer in the parent profile. We call r_{ℓ} the *next-synchronization post-residual* (NSPR) metric because it measures the residual perturbation after that next synchronization and therefore indicates whether later synchronization repairs the omitted update. The metrics u_{ℓ} , o_{ℓ} , and v_{ℓ} protect typical, outlier, and action-level behavior against the fixed reference. We rank each metric independently in ascending order and sum the four ranks. If ℓ has no subsequent synchronization layer, its unavailable NSPR rank

Algorithm 1 NSPR-Guided Sequential Layer Screening

Input: fixed reference $\mathcal{S}_{\text{ref}}$; initial profile $\mathcal{S}_0$; phased replay set $\mathcal{X}$; paired development set $\mathcal{D}$; minimum size m ; removal and candidate budgets R, K ; bootstrap count B ; success tolerance δ .

Output: screened synchronization profile $\mathcal{S}^*$.

```

1:  $\mathcal{S} \leftarrow \mathcal{S}_0$ ;  $n_{\text{drop}} \leftarrow 0$ 
2: cache reference action and intent outputs from
    $\text{Replay}(\mathcal{S}_{\text{ref}}, \mathcal{X})$ 
3: while  $|\mathcal{S}| > m$  and  $n_{\text{drop}} < R$  do
4:   replay the current parent profile  $\mathcal{S}$  on  $\mathcal{X}$ 
5:   for each  $\ell \in \mathcal{S}$  do
6:      $\mathcal{C}_{\ell} \leftarrow \mathcal{S} \setminus \{\ell\}$ ; replay  $\mathcal{C}_{\ell}$  on  $\mathcal{X}$ 
7:     compute  $(r_{\ell}, u_{\ell}, o_{\ell}, v_{\ell})$  against the parent and  $\mathcal{S}_{\text{ref}}$ 
8:   end for
9:    $\mathcal{V} \leftarrow \text{StableTopKCandidates}(\{r_{\ell}, u_{\ell}, o_{\ell}, v_{\ell}\}, \mathcal{X}, B, K)$ 
10:   $\text{accepted} \leftarrow \text{false}$ 
11:  for each  $\ell \in \mathcal{V}$  in ranked order do
12:    evaluate  $\mathcal{S}$  and  $\mathcal{C}_{\ell}$  on identical closed-loop conditions  $\mathcal{D}$ 
13:    if  $P_{\mathcal{D}}(\mathcal{C}_{\ell}) \geq P_{\mathcal{D}}(\mathcal{S}) - \delta$ , no new timeout occurs, and all
       outputs are finite then
14:       $\mathcal{S} \leftarrow \mathcal{C}_{\ell}$ 
15:       $n_{\text{drop}} \leftarrow n_{\text{drop}} + 1$ 
16:       $\text{accepted} \leftarrow \text{true}$ 
17:      break
18:    end if
19:  end for
20:  if not  $\text{accepted}$  then
21:    break
22:  end if
23: end while
24: return  $\mathcal{S}^* \leftarrow \mathcal{S}$ 

```

Lower metric values indicate safer removals; the paired gate can veto but cannot nominate a layer.

is replaced by the mean rank of the other three metrics. Phase-stratified bootstrap resampling then estimates how often a candidate remains among the two safest candidates. Candidate order is determined first by this Top-2 frequency and then by the four-metric rank sum.

C. NSPR-Guided Sequential Search

The next active synchronization layer provides a natural repair opportunity: measuring before it would count perturbations that the system is designed to correct, whereas measuring after it isolates the residual that survives this correction. Even this post-repair metric cannot authorize removal by itself. Fixed-state replay is fast and parallelizable, but it omits the environment feedback through which one action error changes all subsequent observations. Paired closed-loop rollout is more expensive but captures this distribution shift. We therefore use replay metrics only to nominate candidates and reserve the final decision for a behavioral safety gate.

Algorithm 1 iteratively evaluates drop-one candidates on fixed replay states, forwards a small set of stable candidates to paired closed-loop evaluation, and accepts at most one safe removal per iteration. A candidate must remain within the success tolerance, produce finite outputs, and introduce no additional timeout. Screening terminates when no candidate passes this gate or the target layer budget is reached.

V. EXPERIMENTAL EVALUATION

We first evaluate task capability and the necessity of inter-agent communication, and then focus on the main systems

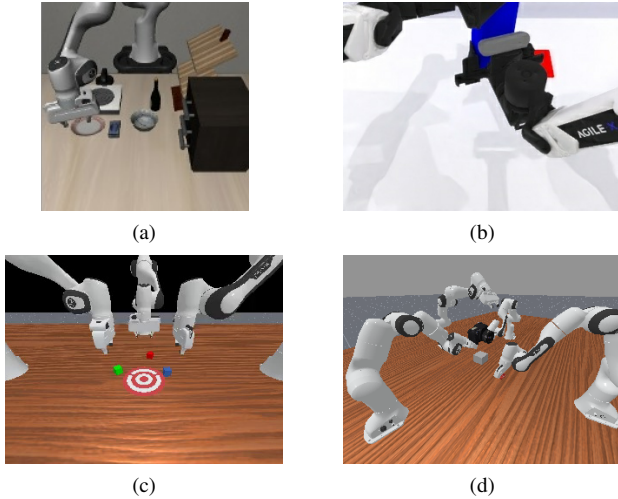


Fig. 3. Representative simulation workloads with (a) one, (b) two, (c) three, and (d) four VLA agents.

questions: how representation, synchronization frequency, and layer screening affect traffic, latency, and closed-loop performance.

A. Experimental Setup

Each agent is initialized from an Eagle2-1B/OpenVLA-style single-arm VLA and predicts a 20-step chunk of 10-D end-effector and gripper actions. Training is centralized with distributed data parallelism; inference launches one process and GPU per agent. All measurements are collected on one server using PyTorch distributed collectives with CPU-mediated tensors, preserving explicit process and device boundaries and measurable data movement between agents. Every planning round records message type, layer, representation, codec, payload, collective count, critical-path communication time, and end-to-end planning time where available.

All formal policy comparisons reuse frozen rollout-condition files and a fixed checkpoint for each completed evaluation. We report paired statistics where applicable. We use *Raw* for the direct operator-level interface, *Full* for the native all-layer hidden-workspace profile, and *NSPR-8* for the screened eight-layer profile. Code, configurations, communication sweeps, and reproducibility manifests are available at <https://anonymous-0730.github.io/VLA-Fabric/>.

B. Simulation Workloads and Capability

Our simulation suite spans single-arm manipulation, strongly coupled bimanual coordination, and collaborative tasks with three and four agents. Figure 3 gives one representative view at each scale, and Table II reports the compact policy comparison. For $N = 1$, VLA-Fabric degenerates to its local SingleVLA policy because no peer exists and communication is not applicable. The SingleVLA results in Table II are reported by the TwinVLA study [9].

For one agent, the inherited SingleVLA policy provides a strong manipulation prior across all four LIBERO suites. At $N = 2$, the centralized TwinVLA reference can combine cross-arm inputs within one computation graph, whereas

TABLE II
TASK SUCCESS RATES (%). FULL USES ALL LAYERS; ROBOFACTORY DP USES 150 DEMONSTRATIONS.

Agents	Task	Method	Success
1	LIBERO-Spatial	SingleVLA	92.4
	LIBERO-Object	SingleVLA	94.5
	LIBERO-Goal	SingleVLA	93.5
	LIBERO-Long	SingleVLA	63.7
2	Handover box	TwinVLA	76.0
		VLA-Fabric, Full	75.0
	Shoes table	TwinVLA	83.0
		VLA-Fabric, Full	78.0
	Handover mic	TwinVLA	94.5
		VLA-Fabric, Full	81.5
3	Camera alignment	RoboFactory DP [16]	19.0
		VLA-Fabric, Full	100.0
	Stack cube	RoboFactory DP [16]	22.0
		VLA-Fabric, Full	75.0
4	Take photo	RoboFactory DP [16]	20.0
		VLA-Fabric, Full	96.5

TABLE III
SUCCESS RATES (%) UNDER COORDINATION-MESSAGE ABLATIONS. FULL DENOTES THE NATIVE ALL-LAYER PROFILE; A DASH DISABLES A MESSAGE AND “RAND.” REPLACES IT WITH A RANDOM TENSOR.

Common	Private K/V	Action	Handover box	Shoes table	Handover mic	Stack cube
✓	✓	✓	75.0	78.0	81.5	75.0
–	✓	✓	38.5	19.0	52.5	66.0
✓	–	✓	59.0	64.5	60.5	50.0
✓	Rand.	✓	62.5	37.5	54.0	0.0
✓	✓	–	43.0	7.5	52.5	29.0
✓	✓	Rand.	31.0	17.0	66.0	26.5
–	–	–	20.0	1.5	13.5	2.0

each VLA-Fabric agent receives only local observations and coordinates through the three named intermediate messages. Under this distributed constraint, VLA-Fabric remains within one and five percentage points of TwinVLA on handover box and shoes table, respectively. At $N = 3$, Full reaches 100.0% on Camera Alignment and 75.0% on Stack Cube, compared with 19.0% and 22.0% for RoboFactory DP trained with 150 demonstrations [16]. At $N = 4$, Full reaches 96.5% on Take Photo, compared with 20.0% for the same baseline. Together, results from $N = 1$ to $N = 4$ demonstrate agent-wise composition without changing local action dimensionality.

C. Coordination Messages

The architecture exposes common semantics, private K/V context, and action intent because they occupy different policy locations and carry different coordination roles. Table III combines paired structural ablations with random-message stress tests across three bimanual tasks and three-agent Stack Cube. The random controls distinguish whether performance depends on message content rather than merely on keeping a channel active.

The all-message reference in Table III is Full: common hidden workspace is synchronized at all 24 layers, with private K/V and action intent enabled. All three paths contribute

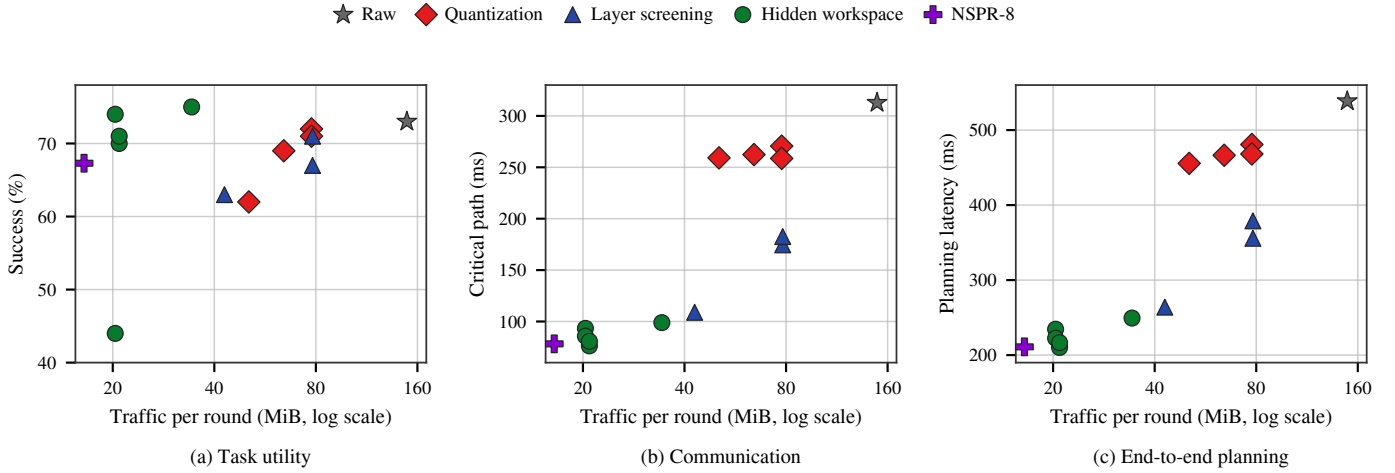


Fig. 4. Communication-efficiency trade-offs. (a) Closed-loop success, (b) communication critical-path latency, and (c) total planning latency versus bidirectional traffic per round. Colors and markers distinguish communication families.

TABLE IV
COMMUNICATION PREPROCESSING AND LAYER-SCREENING RESULTS ON HANDOVER BOX.

Family	Representation	Synchronized layers	Traffic (MiB)	Reduction (%)	Collectives	Success (%)	Critical path (ms)	Planning (ms)
Raw projected	BF16	all 24	148.877	0.00	486	73.0	312.82	538.57
Projected quantization	packed INT8	all 24	77.646	47.85	486	72.0	270.62	480.77
Projected quantization	FP8 E4M3	all 24	77.646	47.85	486	71.0	258.66	468.14
Projected quantization	mixed INT4-A	all 24	64.275	56.83	486	69.0	262.37	466.41
Projected quantization	mixed INT4-B	all 24	50.625	66.00	486	62.0	259.16	455.60
Projected layer screening	BF16	late 12	78.202	47.47	318	67.0	174.57	355.78
Projected layer screening	BF16	even 12	78.202	47.47	318	71.0	182.61	378.98
Projected layer screening	BF16	late-even 6	42.864	71.21	234	63.0	108.95	263.97
Hidden workspace	BF16	all 24	34.268	76.98	198	75.0	98.90	249.39
Hidden workspace	BF16	late 12	20.897	85.96	174	70.0	76.27	210.18
Hidden workspace	BF16	even 12	20.897	85.96	174	71.0	80.73	216.65
Hidden workspace	packed INT8	all 24	20.340	86.34	198	44.0	93.33	234.50
Hidden workspace	FP8 E4M3	all 24	20.340	86.34	198	74.0	85.78	222.53
Hidden workspace + NSPR-8	BF16	selected 8	16.440	88.96	166	67.3	78.25	211.02

independently, with task-dependent effects; action intent is especially important for Shoes table and Stack Cube. Random private K/V is more damaging than omitting that channel on Stack Cube, showing that message content matters. Disabling every path leaves only 1.5–20.0% success across the tabulated tasks, and four-agent Take Photo shows the same dependence. Instrumentation verifies zero bytes for disabled channels and nonzero transmission for random controls. The interface therefore carries task-relevant information rather than serving only as an implementation boundary.

D. Communication-Efficiency Trade-offs

Figure 4 and Table IV compare three complementary ways of reducing communication: lowering representation precision, reducing synchronization frequency, and redesigning the common-path transport boundary. Raw is the direct high-traffic realization of the original cross-agent dependencies. Unless explicitly stated otherwise, every profile retains private K/V and action-intent communication.

Quantization substantially reduces payload but preserves the dense synchronization schedule and its collective-launch,

staging, and coordination overheads. Layer selection removes both bytes and synchronization operations, although projected profiles still communicate several operator tensors at every retained layer. Hidden workspace provides the main structural gain by eliminating both representation and synchronization redundancy. The profiler sums calls across both ranks, and the two interfaces follow

$$N_{\text{coll}}^{\text{proj}}(\mathcal{S}) = 150 + 14|\mathcal{S}|, \quad N_{\text{coll}}^{\text{hidden}}(\mathcal{S}) = 150 + 2|\mathcal{S}|. \quad (16)$$

The fixed term covers communication outside configurable common synchronization; every active projected layer adds 14 calls, whereas a hidden-workspace layer adds only two. BF16 and FP8 hidden profiles consequently preserve task performance while substantially reducing traffic and planning latency. The contrasting hidden INT8 result further shows that codec choice must match representation statistics rather than bit width alone.

NSPR operates after hidden-workspace preprocessing and removes additional synchronization layers according to measured behavioral impact. NSPR-8 reduces communication by

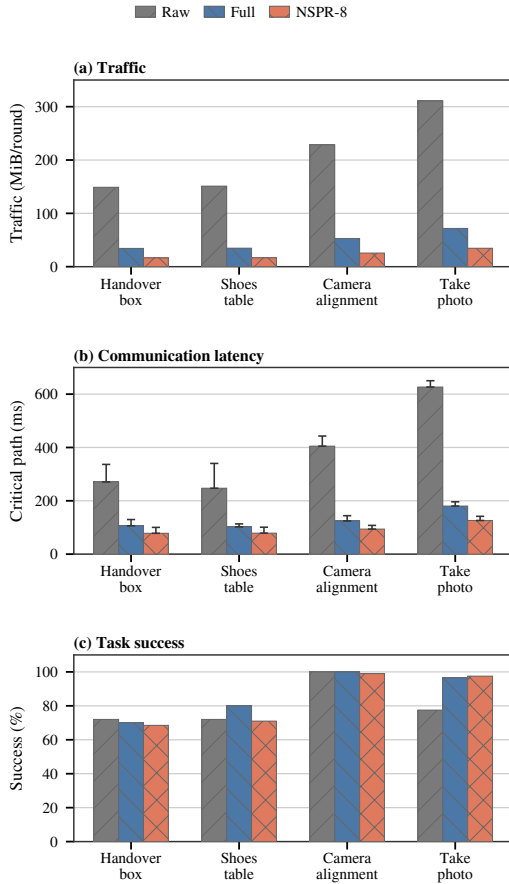


Fig. 5. Cross-task transfer of NSPR-8. Panel (a) reports the fixed bidirectional payload per round, panel (b) reports critical-path mean and P95 from strict timing-50 runs, and panel (c) reports paired-200 success.

88.96% relative to Raw when both profiles are reported in MiB. Across the paired 600-trial evaluation, it reaches 67.3% success, 2.7 percentage points below its 70.0% reference and within the predefined safety tolerance. Overall, communication efficiency depends on jointly reducing payload, synchronization frequency, and representation redundancy rather than lowering precision alone.

E. Screening Stability and Cross-Task Transfer

Figure 5 evaluates the same fixed eight-layer mask without task-specific retuning across two bimanual workloads, three-agent Camera Alignment, and four-agent Take Photo.

Starting from the 12-layer hidden-even profile, three independent screening runs produce one ten-layer and two eight-layer profiles. All three remove layers 8 and 12 and retain the same seven-layer core $\{0, 2, 4, 10, 16, 20, 22\}$. The remaining differences arise only during more aggressive screening, indicating that NSPR repeatedly identifies a stable low-communication region without requiring one unique mask.

Across all four tasks, the fixed profile reduces traffic by 88.8–89.0% relative to Raw without reducing success by more than 3.5 percentage points. Take Photo also confirms that Raw is a high-traffic reference rather than a performance upper bound: its operator-level aggregation differs from the check-

point’s native Full path. NSPR-8 preserves Full performance on this task while reducing both traffic and communication critical path. The fixed mask therefore transfers beyond the screening tasks and remains effective as team size increases.

VI. LIMITATIONS AND FUTURE DIRECTIONS

VLA-Fabric decouples local action dimensionality from team size, although the current role-ordered all-to-all topology still creates $N(N - 1)$ directed peer relations under unicast transport. This topology provides direct and symmetric coordination for the small, tightly coupled teams studied here, but its pairwise traffic grows quadratically as agents are added. Because communication is exposed as a graph rather than embedded inside a centralized policy, further work can investigate sparse or adaptive topologies and channel scheduling according to task structure, execution phase, and available network resources. Future work could extend the same interface to heterogeneous agents, physical links, and real robots.

Despite replacing centralized joint control with agents that observe only local inputs, VLA-Fabric remains close to centralized references on the ALOHA tasks. This result indicates that agent-wise composition can provide substantial architectural extensibility without requiring a correspondingly large loss in task capability. The remaining performance gap is not an intrinsic limit of the organization: stronger local policies, improved cross-agent representation fusion, and communication-aware training can be incorporated without changing the one-arm-one-agent interface. More broadly, making cross-policy messages explicit and measurable creates a foundation for communication research in embodied AI and connects networking advances with robotic foundation models.

VII. CONCLUSIONS

Centralized multi-arm VLA policies bind coordination, action dimensionality, and deployment to one fixed computation graph. In this paper, we proposed VLA-Fabric, a distributed organization in which one complete VLA agent controls each arm, predicts only a local action, and coordinates with peers through three explicit intermediate-representation messages. We further developed communication preprocessing and NSPR-guided layer screening to reduce redundant exchange while protecting closed-loop behavior. Experiments show that all three messages contribute to coordination, distributed policies remain close to centralized references on the ALOHA tasks, and communication can be reduced by up to 89.0% relative to Raw. The screened profile transfers across two- to four-agent tasks while maintaining closed-loop success. These results establish communication-efficient intermediate-representation exchange as a practical systems interface for scalable VLA agents and a step toward a broader embodied intelligence fabric.

REFERENCES

- [1] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia *et al.*, “RT-2: Vision-language-action models transfer web knowledge to robotic control,” in *Proceedings of the 7th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 229, 2023, pp. 2165–2183.
- [2] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Raffailov, E. P. Foster *et al.*, “OpenVLA: An open-source vision-language-action model,” in *Proceedings of the 8th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 270, 2025, pp. 2679–2713.
- [3] D. Ghosh, H. R. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, T. Kreiman *et al.*, “Octo: An open-source generalist robot policy,” in *Proceedings of Robotics: Science and Systems*, 2024.
- [4] K. Black, N. Brown, D. Driess, A. Esmail, M. R. Equi, C. Finn, N. Fusai, L. Groom *et al.*, “ π_0 : A vision-language-action flow model for general robot control,” in *Proceedings of Robotics: Science and Systems*, 2025.
- [5] Xiaomi Robotics Team, J. Guo, P. Jin, J. Li, P. Li, Y. Li, F. Liu, W. Peng, O. Qin, Y. Su *et al.*, “Xiaomi-Robotics-1: Scaling Vision-Language-Action models with over 100K hours of real-world trajectories,” *arXiv preprint arXiv:2607.15330*, 2026.
- [6] D. Qu, H. Song, Q. Chen, Y. Yao, X. Ye, J. Gu, Z. Wang, Y. Ding, B. Zhao, D. Wang, and X. Li, “SpatialVLA: Exploring spatial representations for Visual-Language-Action models,” in *Proceedings of Robotics: Science and Systems*, 2025.
- [7] K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. R. Equi, C. Finn, N. Fusai, M. Y. Galliker, D. Ghosh, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, D. LeBlanc, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, A. Z. Ren, L. X. Shi, L. Smith, J. T. Springenberg, K. Stachowicz, J. Tanner, Q. Vuong, H. Walke, A. Walling, H. Wang, L. Yu, and U. Zhilinsky, “ $\pi_{0.5}$: A vision-language-action model with open-world generalization,” in *Proceedings of the 9th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 305, 2025, pp. 17–40.
- [8] C. Li and Y. Peng, “Embodiment transfer learning for vision-language-action models,” *arXiv preprint arXiv:2511.01224*, 2025.
- [9] H. Im, E. Jeong, A. Kolobov, J. Fu, and Y. Lee, “TwinVLA: Data-efficient bimanual manipulation with twin single-arm vision-language-action models,” in *International Conference on Learning Representations*, 2026.
- [10] S. Sukhbaatar, A. Szlam, and R. Fergus, “Learning multiagent communication with backpropagation,” in *Advances in Neural Information Processing Systems*, vol. 29, 2016, pp. 2244–2252.
- [11] J. N. Foerster, Y. M. Assael, N. de Freitas, and S. Whiteson, “Learning to communicate with deep multi-agent reinforcement learning,” in *Advances in Neural Information Processing Systems*, vol. 29, 2016, pp. 2137–2145.
- [12] A. Das, T. Gervet, J. Romoff, D. Batra, D. Parikh, M. Rabbat, and J. Pineau, “TarMAC: Targeted multi-agent communication,” in *Proceedings of the 36th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 97, 2019, pp. 1538–1546.
- [13] G. Li, H. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem, “CAMEL: Communicative agents for “mind” exploration of large language model society,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 51 991–52 008.
- [14] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, “AutoGen: Enabling next-gen LLM applications via multi-agent conversations,” in *First Conference on Language Modeling*, 2024.
- [15] J. Chen, J. Li, Y. Huang, C. Garrett, D. Sun, C. Fan, A. Hofmann, C. Mueller, S. Koenig, and B. C. Williams, “Cooperative task and motion planning for multi-arm assembly systems,” *arXiv preprint arXiv:2203.02475*, 2022.
- [16] Y. Qin, L. Kang, X. Song, Z. Yin, X. Liu, X. Liu, R. Zhang, and L. Bai, “RoboFactory: Exploring embodied agent collaboration with compositional constraints,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2025, pp. 10 075–10 085.
- [17] T. Chen, Z. Chen, B. Chen, Z. Cai, Y. Liu, Z. Li, Q. Liang, X. Lin, Y. Ge, Z. Gu *et al.*, “RoboTwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation,” in *International Conference on Machine Learning*, 2026.
- [18] X. Liu, L. Zhang, D. Ganesan, and H. Guan, “Communication-efficient multi-device inference acceleration for transformer models,” *arXiv preprint arXiv:2505.19342*, 2025.
- [19] S. Liu, L. Wu, B. Li, H. Tan, H. Chen, Z. Wang, K. Xu, H. Su, and J. Zhu, “RDT-1B: A diffusion foundation model for bimanual manipulation,” in *International Conference on Learning Representations*, 2025.
- [20] Y. Huang, Y. Cheng, A. Bapna, O. Firat, M. X. Chen, D. Chen, H. Lee, J. Ngiam, Q. V. Le, Y. Wu, and Z. Chen, “GPipe: Efficient training of giant neural networks using pipeline parallelism,” in *Advances in Neural Information Processing Systems*, vol. 32, 2019, pp. 103–112.
- [21] D. Narayanan, M. Shoeybi, J. Casper, P. LeGresley, M. Patwary, V. A. Korthikanti, D. Vainbrand, P. Kashinkunti, J. Bernauer, B. Catanzaro, A. Phanishayee, and M. Zaharia, “Efficient large-scale language model training on GPU clusters using Megatron-LM,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2021, pp. 1–15.